

Databases

Interview Cracker question bank

Generated from the Markdown sources in the Interview Cracker repository. Every code snippet in these pages was executed and checked before publication.

Covering Indexes

[Medium] Q. What is a covering index, and when can it make a query much faster?

Answer. A covering index contains every column a query needs for filtering, joining, and output, so the database can answer from the index without looking up the base table rows.

For `SELECT created_at FROM orders WHERE customer_id = ?`, an index on `(customer_id, created_at)` may cover the query. It avoids a random table-page lookup for every match and can be a large win for selective, read-heavy paths.

It is not free: extra columns enlarge the index, reduce cache density, and make inserts and updates more expensive. Verify an index-only scan in the query plan, and remember that MVCC visibility rules may still force table access.

Database Normalization

Normalization is about arranging tables so that each **fact** is stored in exactly one place. The payoff is not academic purity - it is the elimination of update, insertion, and deletion **anomalies**, where one logical change has to be written to (or breaks) several rows.

[Easy] Q. What is a functional dependency, and why is it the root of normalization?

Answer. A functional dependency $x \rightarrow y$ means the value of x determines the value of y - two rows with the same x must have the same y . Normalization is the process of splitting tables so that every non-key attribute depends on the **whole key, nothing but the key, and nothing but the key** (the 3NF slogan). Every normal form is a rule about which dependencies are allowed to live in a table.

```
# An unnormalized "enrollment" table where the teacher is repeated per row.
rows = [
    ("S1", "Math", "Alice"),
    ("S1", "Bio", "Bob"),
    ("S2", "Math", "Alice"),    # "Math is taught by Alice" stored TWICE
]
teachers_for_math = [r[2] for r in rows if r[1] == "Math"]
assert teachers_for_math == ["Alice", "Alice"]    # redundant copies
```

The fact "Math is taught by Alice" depends only on `course`, not on the `(student, course)` key - that is the dependency that normalization will eventually force out of the table.

[Medium] Q. Walk through 1NF, 2NF, and 3NF, and name the anomaly each one removes.

Answer.

- **1NF** - atomic values only, no repeating groups / nested arrays.
- **2NF** - 1NF + no partial dependency: every non-key attribute depends
- **3NF** - 2NF + no transitive dependency: non-key attributes depend on

```
import sqlite3

con = sqlite3.connect(":memory:")
cur = con.cursor()
# Unnormalized: (student, course) key, teacher transitively dependent.
cur.execute(
    "CREATE TABLE enrollment "
    "(id INTEGER PRIMARY KEY, student TEXT, course TEXT, teacher TEXT)"
)
cur.executemany(
    "INSERT INTO enrollment (student, course, teacher) VALUES (?, ?, ?)",
    [("S1", "Math", "Alice"), ("S1", "Bio", "Bob"), ("S2", "Math", "Alice")]
)
# The "update anomaly": one logical fact lives in two rows.
cur.execute("SELECT COUNT(*) FROM enrollment WHERE teacher = 'Alice'")
assert cur.fetchone()[0] == 2

# 3NF: split out the transitive dependency course -> teacher.
cur.execute("CREATE TABLE course (course TEXT PRIMARY KEY, teacher TEXT)")
cur.execute("CREATE TABLE takes (student TEXT, course TEXT, PRIMARY KEY (student, course))")
cur.execute("INSERT INTO course VALUES ('Math', 'Alice'), ('Bio', 'Bob')")
cur.execute("INSERT INTO takes VALUES ('S1', 'Math'), ('S1', 'Bio'), ('S2', 'Math')")
cur.execute()
```

```

"SELECT t.student, t.course, c.teacher "
"FROM takes t JOIN course c ON t.course = c.course ORDER BY 1,2"
)
assert cur.fetchall() == [
    ("S1", "Bio", "Bob"), ("S1", "Math", "Alice"), ("S2", "Math", "Alice"),
]

```

Now "Math is taught by Alice" is stored **once** in `course`; updating it touches one row, and enrolling a student never forces you to restate the teacher.

[Medium] Q. What is BCNF, and how is it stronger than 3NF?

Answer. BCNF says: for every non-trivial dependency $X \rightarrow Y$, X must be a **superkey**. 3NF allows a non-key to determine another non-key as long as the dependent side is a key; BCNF forbids it entirely. In practice the two coincide unless you have overlapping candidate keys.

The classic 3NF-but-not-BCNF example: a table `student, course, teacher` with candidate keys `(student, course)` and `(course, teacher)`, where `course → teacher`. Here `teacher` is part of a candidate key, so 3NF is satisfied (the dependent attribute is a key attribute), but `course → teacher` has a non-superkey determinant, so BCNF is violated. Splitting on `course → teacher` (a `course(teacher)` table) restores BCNF.

The interview takeaway: **3NF is the usual stopping point for OLTP schemas** because BCNF decompositions can sometimes lose a dependency and force joins; you reach for BCNF when you specifically want to kill every remaining redundancy.

[Medium] Q. If normalization is so good, why do people denormalize at all?

Answer. Normalization optimises for **write** consistency (no anomalies, minimal storage). Denormalization trades that away to optimise **read** cost - fewer joins, faster reports, simpler queries. It is a deliberate, documented decision, not laziness.

Reason to denormalize	Example
Join-heavy read path is the bottleneck	Store <code>order.total</code> instead of summing <code>line_item</code> each time
Historical / audit value	Keep <code>customer.name_at_purchase</code> frozen at the moment of sale
Redundant fact is cheap and almost never changes	Store <code>country</code> next to a normalised <code>region_id</code>
Analytics / reporting on a copy	A warehouse or materialised view is a denormalised projection

The discipline: identify the specific read you are optimising, store the redundant value, and own the **write path** that keeps it in sync (trigger, application code, or a rebuild job). The moment you denormalise, you re-introduce an update anomaly on purpose - so you must have a mechanism and a test that proves the copy stays consistent.

[Easy] Q. What is a surrogate key vs a natural key, and which do you use?

Answer. A **natural key** is a real-world identifier (email, VIN, passport number); a **surrogate key** is an artificial, meaningless identifier (auto-increment int, UUID) whose only job is to be unique and stable. Surrogates are preferred for primary keys because natural keys change (someone updates their email), are often long (bad for index size and join speed), and can collide or be reused across contexts.

The rule of thumb: use a **surrogate** primary key, keep the natural identifier as a **unique, indexed** column, and foreign keys reference the surrogate. This keeps joins cheap and insulates your schema from the day the

business decides the "real" identifier was a bad choice.

Indexing and Transactions

The two database topics that separate "can write a query" from "can run a system".

[Easy] Q. What is an index and what does it cost?

Answer. A separate data structure - almost always a **B+ tree** - that maps column values to row locations, so the engine can seek instead of scanning every row.

It turns an $O(n)$ table scan into roughly $O(\log n)$.

The costs, which matter as much as the benefit:

- **Writes get slower.** Every `INSERT`, `UPDATE` and `DELETE` must also update
- **Storage.** An index on a wide column can approach the size of the table.
- **They can be ignored.** If a query would match most of the table, the planner

So: index columns you filter, join or sort on frequently. Do not index everything.

[Medium] Q. Why a B+ tree rather than a hash index or a binary tree?

Answer. Because disk and page size dominate.

A B+ tree node holds hundreds of keys and fills one page, so the tree is very shallow - three or four levels indexes millions of rows, meaning three or four page reads. A binary tree with the same data would be ~20 levels deep and each level is a potential random I/O.

B+ trees also keep all data in the **leaves, linked in sorted order**, which makes range scans (`WHERE date BETWEEN ...`, `ORDER BY`) a straight walk along the leaf chain.

A **hash index** gives $O(1)$ equality lookup but supports no range queries, no sorting and no prefix matching. Some engines offer them; the B-tree is the default because it is good at everything rather than excellent at one thing.

[Medium] Q. What is a composite index, and why does column order matter?

Answer. An index on several columns, sorted by the first column, then the second within that, and so on - like a phone book sorted by surname then forename.

This is the **leftmost prefix rule**. With `INDEX (last_name, first_name)`:

Query	Uses the index?
<code>WHERE last_name = 'Khan'</code>	[yes] Yes
<code>WHERE last_name = 'Khan' AND first_name = 'Sara'</code>	[yes] Yes
<code>WHERE first_name = 'Sara'</code>	[no] No

You cannot look someone up by forename in a phone book; same reason. Put the column you always filter on first, and the highest-selectivity column early.

[Medium] Q. Name things that stop an index being used.

Answer. Mostly, wrapping the indexed column in something:

- **A function on the column.** `WHERE YEAR(created_at) = 2026` cannot use an
- **A leading wildcard.** `LIKE '%son'` cannot seek; `LIKE 'John%'` can.
- **Implicit type conversion.** Comparing an indexed `VARCHAR` to a number makes
- **Low selectivity.** An index on a boolean that is 90% true will be skipped.
- **OR across different columns.** Often forces a scan; `UNION` of two indexed

Use `EXPLAIN` (or `EXPLAIN ANALYZE`) to check rather than guessing - look for `Seq Scan / type: ALL` where you expected a seek.

[Easy] Q. What does ACID stand for?

Answer.

- **Atomicity** - all of a transaction's changes apply, or none do.
- **Consistency** - a transaction moves the database from one valid state to
- **Isolation** - concurrent transactions do not see each other's partial work.
- **Durability** - once committed, changes survive a crash (via the write-ahead

Atomicity is the one people can illustrate: a bank transfer debits one account and credits another, and a crash in between must not leave money destroyed.

[Hard] Q. Explain the isolation levels and the anomalies they prevent.

Answer. Stronger isolation removes more anomalies and costs more concurrency.

Level	Dirty read	Non-repeatable read	Phantom read
Read uncommitted	possible	possible	possible
Read committed	prevented	possible	possible
Repeatable read	prevented	prevented	possible*
Serializable	prevented	prevented	prevented

The anomalies:

- **Dirty read** - you read another transaction's uncommitted change, which may
- **Non-repeatable read** - you read the same **row** twice and get different
- **Phantom read** - you run the same **query** twice and get different rows,

* PostgreSQL's repeatable read uses snapshot isolation and does prevent phantoms; the SQL standard does not require this. Defaults differ too - PostgreSQL and Oracle default to read committed, MySQL/InnoDB to repeatable read. Knowing that defaults vary is a better answer than reciting one vendor's.

[Medium] Q. What is a deadlock and how do you handle it?

Answer. Two transactions each hold a lock the other needs, so neither can proceed. Transaction A locks row 1 and wants row 2; B locks row 2 and wants row 1.

Databases **detect** deadlocks (a cycle in the wait-for graph) and resolve them by killing one transaction - the "victim" - which then gets an error and must retry.

Prevention, in order of usefulness:

- 1 **Consistent lock ordering.** If every transaction touches rows in ascending
 - 2 **Keep transactions short.** Never hold a lock across a network call or user
 - 3 **Lower isolation** where the workload permits it.
 - 4 **Retry with backoff.** Deadlocks are normal under load; application code
-

[Medium] Q. Optimistic vs pessimistic locking?

Answer.

Pessimistic - take the lock before reading (`SELECT ... FOR UPDATE`) and hold it until commit. Nobody else can interfere. Correct, but it serialises access and risks deadlocks. Right when contention is high and conflicts are likely.

Optimistic - take no lock. Read a version number with the row, and on write check the version is unchanged:

```
UPDATE accounts SET balance = 500, version = version + 1
WHERE id = 7 AND version = 3;          -- 0 rows updated means someone else won
```

If zero rows are affected, someone else committed first; re-read and retry. Right when conflicts are rare - no locks, better throughput, at the cost of occasional wasted work.

[Medium] Q. Normalisation vs denormalisation?

Answer. Normalisation removes redundancy by splitting data into related tables. Each fact lives in exactly one place, so updates cannot leave contradictory copies. The cost is more joins.

Denormalisation deliberately duplicates data to avoid those joins - faster reads, but every copy must be kept in sync, and that synchronisation is where bugs live.

The practical answer: **normalise by default, denormalise with evidence.** Start at third normal form, measure, and denormalise the specific read path that is demonstrably too slow. Analytics and reporting systems (star schemas) are routinely denormalised because they are read-heavy and loaded in batches.

Replication Lag

[Hard] Q. A user writes data, refreshes, and cannot see it. How can replication cause this?

Answer. If writes go to a primary but the refresh reads from an asynchronous replica, that replica may not have replayed the write yet. This violates read-your-writes consistency even though replication is healthy.

Remedies include routing that user's reads to the primary briefly, carrying the write's log position and waiting for a caught-up replica, or using synchronous replication when visibility is worth higher latency and lower availability. A UI can also acknowledge pending processing.

Monitoring only average lag is insufficient: alert on high percentiles and log positions behind. A replica can be seconds behind during a burst and look normal again by the time a coarse dashboard samples it.

SQL Query Questions

SQL rounds are mostly the same eight ideas. Know joins, grouping, window functions and how NULL behaves, and you will handle almost anything asked.

[Easy] Q. Explain the join types.

Answer. A join matches rows from two tables on a condition; the type decides what happens to rows that find no match.

Join	Keeps
INNER	Only rows matching in both
LEFT	All left rows; NULLs where the right has no match
RIGHT	All right rows; NULLs where the left has no match
FULL OUTER	All rows from both sides
CROSS	Every combination (Cartesian product)

The **anti-join** is the pattern worth having ready - "rows on the left with no match on the right":

```
SELECT c.*
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NULL;      -- the join produced no match
```

Worked examples: [examples/sql/joins/inner_left_right_and_anti_joins.sql](#).

[Medium] Q. What is the difference between WHERE and HAVING?

Answer. WHERE filters rows before grouping. HAVING filters groups after aggregation.

```
SELECT department, COUNT(*) AS headcount
FROM employees
WHERE active = TRUE          -- drop inactive people first
GROUP BY department
HAVING COUNT(*) > 5;         -- then drop small departments
```

You cannot put `COUNT(*) > 5` in WHERE, because at that point the groups do not exist yet. The logical order of evaluation explains every question of this shape:

```
FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT
```

SELECT running so late is also why you generally cannot reference a column alias in WHERE, but usually can in ORDER BY.

[Medium] Q. What is the trap with NULL?

Answer. NULL means unknown, not empty, and comparisons with unknown are themselves unknown - not false.

- `NULL = NULL` is **NULL**, not true. Use `IS NULL`.
- `WHERE status != 'shipped'` **excludes rows where status is NULL**, which
- Aggregates skip NULLs: `COUNT(col)` ignores them, `COUNT(*)` does not. `AVG`

The worst one, and a genuine favourite in interviews:

```
SELECT * FROM employees
WHERE id NOT IN (SELECT manager_id FROM departments);
```

If any `manager_id` is `NULL`, this returns **zero rows**, always. `x NOT IN (1, NULL)` evaluates to `NOT (x=1 OR x=NULL) = NOT (false OR unknown) = unknown`. Use `NOT EXISTS`, which handles `NULL` correctly. See [examples/sql/subqueries/exists_in_and_correlated.sql](#).

[Medium] Q. Find the second-highest salary.

Answer. Several ways; the interviewer usually wants to see you handle ties and the "no such row" case.

```
-- Ranking approach: DENSE_RANK treats tied salaries as one rank.
SELECT DISTINCT salary
FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
      FROM employees) ranked
WHERE rnk = 2;

-- Without window functions:
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

The second form returns `NULL` rather than no rows when everyone earns the same - worth saying out loud, because "what if there is no second-highest?" is the follow-up.

[Medium] Q. Explain RANK, DENSE_RANK and ROW_NUMBER.

Answer. All three number rows within a window; they differ on ties.

For salaries 100, 90, 90, 80:

Salary	ROW_NUMBER	RANK	DENSE_RANK
100	1	1	1
90	2	2	2
90	3	2	2
80	4	4	3

- `ROW_NUMBER` - always distinct, ties broken arbitrarily.
- `RANK` - ties share a rank, then it **skips**.
- `DENSE_RANK` - ties share a rank, no gaps.

Use `ROW_NUMBER` for deduplication (keep the first row per group), `DENSE_RANK` for "top 3 salary levels". Worked example: [examples/sql/window-functions/rank_vs_dense_rank_vs_row_number.sql](#).

[Medium] Q. How do you remove duplicate rows, keeping the newest per group?

Answer. Number rows within each group, then keep number 1.

```
WITH ranked AS (
  SELECT id, email, updated_at,
         ROW_NUMBER() OVER (PARTITION BY email ORDER BY updated_at DESC) AS rn
  FROM users
)
DELETE FROM users
WHERE id IN (SELECT id FROM ranked WHERE rn > 1);
```

The CTE is required because a window function cannot appear in `WHERE` - `WHERE` is evaluated before window functions are. Full example: examples/sql/window-functions/deduplicate_keep_newest_per_group.sql.

[Medium] Q. What is a CTE, and when would you use a recursive one?

Answer. A Common Table Expression is a named subquery defined with `WITH`, used to give a step a name and to reference it more than once. It makes long queries readable and is often the only way to use a window function in a filter.

A **recursive** CTE references itself, which is how you traverse hierarchies - org charts, category trees, bill-of-materials, graph reachability:

```
WITH RECURSIVE chain AS (  
    SELECT id, name, manager_id, 1 AS depth  
    FROM employees WHERE manager_id IS NULL      -- anchor  
    UNION ALL  
    SELECT e.id, e.name, e.manager_id, c.depth + 1  
    FROM employees e JOIN chain c ON e.manager_id = c.id  -- recursive step  
)  
SELECT * FROM chain ORDER BY depth;
```

Always confirm the recursion terminates - a cycle in the data loops forever unless you track visited nodes or cap the depth. Worked example: examples/sql/recursive-ctes/org_chart_tree_traversal.sql.

[Medium] Q. What is the difference between `UNION` and `UNION ALL`?

Answer. `UNION` removes duplicate rows; `UNION ALL` keeps everything.

Deduplication requires a sort or hash over the combined result, so `UNION` is materially more expensive. **Use `UNION ALL` unless you actually need distinct rows** - this is one of the most common easy wins in slow queries.